2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)
October 11-14, 2020. Toronto, Canada

UHUOPM: High Utility Occupancy Pattern Mining in Uncertain Data

Chien-Ming Chen[1], Lili Chen[1], Wensheng Gan[2]*, Lina Qiu[3] and Weiping Ding[4]
[1]*Shandong University of Science and Technology, Shandong, China*
[2]*Jinan University, Guangdong, China*
[3]*South China Normal University, Guangdong, China*
[3]*Nantong University, Jiangsu, China*
Email: chienmingchen@ieee.org, lilichien3@gmail.com, wsgan001@gmail.com,
lina.qiu@scnu.edu.cn, dwp9988@163.com

***Abstract*— It is widely known that there is a lot of useful information hidden in big data, and it is prevalent for individuals to mine crucial information for utilization in many real-world applications. To find patterns that can represent the supporting transaction, a recent study was conducted to mine high-utility occupancy patterns whose contribution to the utility of the entire transaction is greater than a certain value. Moreover, in realistic applications, patterns may not exist in transactions but be connected to an existence probability. In this paper, a novel algorithm, called High Utility-Occupancy Pattern Mining in Uncertain databases (UHUOPM), is proposed. The patterns found by this algorithm are called Potential High Utility Occupancy Patterns (PHUOPs). To reduce memory cost and time consumption and to prune the search space in the algorithm as mentioned above, probability-utility-occupancy list (PUO-list) and probability-frequency-utility table (PFU-table) are used. Finally, substantial experiments were conducted to evaluate the performance of proposed UHUOPM algorithm on both real-life and synthetic datasets.**

I. Introduction

In recent decades, a multitude of investigators have hastened to improve data mining algorithms [1], [2], [3] due to only finding limited interestingness measures, for example, frequency or support. However, these are insufficient because every item is actually unequal. Other preferences, like the profit and risk [4], [5], have increasingly been studied. High-utility itemset mining (HUIM) algorithms [6], [7], [8] consider the unit utility and quantity of objects or items. Liu *et al.* [9] designed the two-phase model in which the main emphasis is to find the upper bound of the utility of itemsets and then trim most of the unqualified itemsets without further calculating their supersets. First, Tang *et al.* [10] creatively proposed the concept of occupancy, which is the ratio of the number of items in the itemset to the one in the transaction. Unfortunately, it could not help solve the problem of profit and utility. To address this issue, Shen *et al.* [11] proposed the concepts of occupancy and utility and presented utility occupancy with the OCEAN algorithm, which is used to find patterns on behalf of the relative supporting transactions in the utility ratio. However, this algorithm does not completely find the patterns that meet the requirements. Gan *et al.* [12] put forward a novel algorithm, called high-utility occupancy pattern mining (HUOPM) algorithm. It can avoid the errors in the OCEAN algorithm [11] and uses two novel structures to prune the search space.

The algorithms mentioned above are all based on precise databases. Conversely, as a result of noisy data sources or failure of data transmission, losing some data is unavoidable [13]. Therefore, it is difficult to apply these existing algorithms to handle uncertain database. Uapriori [14] was the first algorithm to discover frequent itemsets from uncertain databases. It mainly adopts a generate-and-test mechanism. Subsequently, and an approach without generating candidates by utilizing a UFP-tree structure was introduced [15]. Utility-based mining in uncertain databases is also indispensable. For example, PHUI-UP [16] algorithm and PHUI-List algorithm [16] were designed to seek out potential high-utility patterns in uncertain databases. PHUI-UP adopts an upper bound strategy, depends on multiple scanning the database, and generates a large number of candidate itemsets during the mining process. PHUI-List uses a vertical structure to store data, and the pruning strategies mentioned in this algorithm can speed up the mining process.

So far, decision-makers can not find out existing algorithms for analyzing some complicated data with uncertainty and revealing high utility-occupancy patterns. In this paper, we focus on the need for combining utility occupancy and uncertain databases, and we introduce a novel algorithm, called High Utility-Occupancy Pattern Mining in Uncertain databases (UHUOPM). Three factors are involved in this algorithm, namely frequency, probability, and utility occupancy. Moreover, several pruning strategies are proposed to reduce the search space. To evaluate the performance of the compared algorithms, subsequent experiments were conducted on both real-life and synthetic datasets. The experiments show that several pruning strategies can effectively eliminate most of the unqualified patterns and improve the performance of the designed algorithm in terms of memory consumption and runtime. To the best of our knowledge, this is the first study to address the problem of utility-driven exploiting high utility-occupancy patterns in uncertain data.

II. Preliminary and Problem Statement

The uncertain database is shown in Table I, and it is made up of ten transactions and five items that are distinct to each other. Let $I = \{i_1, i_2, \ldots, i_m\}$ be a collection of items; and let $D = \{T_1, T_2, \ldots, T_n\}$ be an uncertain quantitative database, where in supporting transactions, such as T_k. And each

Authorized licensed use limited to: Rutgers University. Downloaded on May 20,2021 at 01:59:59 UTC from IEEE Xplore. Restrictions apply.

item i_c consists of three parts: the item name, the number of occurrences $q(i_c, T_k)$, and the corresponding probability of occurrence $p(i_c, T_k)$. The sum of the utility of each transaction is *tu*. Table II shows the unit utility of each item, which are manually defined. Table I and Table II are taken as an example to explain the proposed algorithm below.

TABLE I: Example of an uncertain quantitative database

tid	Transaction (item, quantity, probability)	*tu*
T_1	{*a*:2, 0.6} {*c*:4, 0.8} {*d*:7, 0.5}	$65
T_2	{*b*:2, 0.7} {*c*:3, 0.4}	$37
T_3	{*a*:3, 0.6} {*b*:2, 0.6} {*c*:1, 0.9} {*d*:2, 0.8}	$38
T_4	{*b*:4, 0.5} {*d*:3, 0.8}	$11
T_5	{*a*:1, 0.9} {*b*:3, 0.7} {*c*:2, 0.9} {*d*:5, 0.6} {*e*:1, 0.8}	$49
T_6	{*c*:2, 0.9} {*e*:4, 0.8}	$58
T_7	{*c*:2, 0.4} {*d*:1, 0.9}	$23
T_8	{*a*:3, 0.6} {*b*:1, 0.8} {*d*:2, 0.8} {*e*:4, 0.5}	$61
T_9	{*a*:2, 0.6} {*c*:4, 0.5} {*d*:1, 0.3}	$59
T_{10}	{*c*:3, 0.6} {*e*:1, 0.7}	$42

TABLE II: Unit utility of each item

Item	*a*	*b*	*c*	*d*	*e*
Utility ($)	7	2	11	1	9

Definition 1: Supposing that in a given database, itemset X appears in these transactions, then the number of occurrences is called support count (SC) and denoted as $SC(X)$ [1], [17]. Then, the transactions that meet the conditions are put into a collection Γ_X, and thus the equation $SC(X) = |\Gamma_X|$ can be obtained. X is considered as a frequent pattern if and only if $SC(X)$ is equal to or greater than a predefined minimum support threshold α.

Example 1: In Table I, it can be seen that pattern (b) appears in transaction T_2, T_3, T_4, T_5, and T_8, respectively. Therefore, it can be concluded as $SC(b) = 5$.

Definition 2: As shown in Table II, each item corresponds to a certain unit utility in the database. If $p(i_c)$ denotes the unit utility of each item, then the utility of the pattern $u(i_c, T_k) = p(i_c) \times q(i_c, T_k)$, where item i_c exists in the transaction T_k. The utility of itemset X in a supporting transaction can be expressed as $u(X, T_k) = \sum_{i_j \in X \wedge X \subseteq T_k} u(i_j, T_k)$. Moreover, the utility of X in a given database D is defined as $u(X) = \sum_{X \subseteq T_q \wedge T_q \in D} u(X, T_q)$. Finally, the transaction utility (tu) is defined as the sum of the utilities of all the items in this transaction.

Example 2: For example, $u(bc) = u(bc, T_2) + u(bc, T_3) + u(bc, T_5)$ = \$37 + \$15 + \$28 = \$80. Thus, $tu(T_1) = u(a, T_1) + u(c, T_1) + u(d, T_1)$ = \$14 + \$44 + \$7 = \$65.

Definition 3: In a transaction database, the utility contribution rate of an itemset in a database is also very significant, which is called the utility occupancy [12]. The utility occupancy of an itemset X in the relative supporting transaction T_k is expressed as:

$$uo(X, T_k) = \frac{u(X, T_k)}{tu(T_k)}. \tag{1}$$

Like the definition of utility, the calculation formula of utility occupancy of itemset X in a database is defined as:

$$uo(X) = \frac{\sum_{X \subseteq T_k \wedge T_k \in D} uo(X, T_k)}{|\Gamma_X|}, \tag{2}$$

where Γ_X is a collection of transactions containing itemset X, and $|\Gamma_X|$ is the length of the collection.

Example 3: According to Definition 2, it can be obtained that $tu(T_2)$ = \$37, $tu(T_3)$ = \$38, $tu(T_4)$ = \$11, $tu(T_5)$ = \$49, and $tu(T_8)$ = \$61. Therefore, it is simple to calculate $uo(b)$, which is first to compute the value of $u(b, T_2)/tu(T_2) + u(b, T_3)/tu(T_3) + u(b, T_4)/tu(T_4) + u(b, T_5)/tu(T_5) + u(b, T_8)/tu(T_8)$, and then divide this value by 5. Finally, it can be calculated that the final result is approximately 0.2257.

Definition 4: This paper focuses on situations with uncertain databases, where the probability of data is used to represent the uncertainty. Let $pro(X)$ represents the probability of itemset X in the corresponding transaction, and it can be denoted as $pro(X) = \sum_{i=1}^{|D|} \prod_{x_i \in X} p(x_i, T_k)$ [16]. In the UHUOPM model, the possibility of a pattern in a given database is defined as $pro(X) = \sum_{X \subseteq T_k \wedge T_k \in D} p(X, T_k)$.

Example 4: For example, $pro(b) = p(b, T_2) + p(b, T_3) + p(b, T_4) + p(b, T_5) + p(b, T_8)$ = 0.7 + 0.6 + 0.5 + 0.7 + 0.8 = 3.3, and $pro(bc) = p(bc, T_2) + p(bc, T_3) + p(bc, T_5) + p(bc, T_8)$ = 0.28 + 0.54 + 0.63 = 1.45.

Definition 5: Given an uncertain database, if the support of an itemset X is no less than the predefined minimum support threshold α, the utility occupancy is no less than the minimum utility occupancy threshold β, and the probability is equal to or greater than the predefined minimum probability threshold γ, then it can be called a potential high utility-occupancy pattern (*PHUOP*). These thresholds are flexibly set according to the requirements of decision-makers based on their prior knowledge and interest.

Example 5: It is not difficult to obtain that $SC(b)$ = 5, $SC(bc)$ = 3, $uo(b)$ = 0.2257, $uo(bc)$ = 1.6553, $pro(b)$ = 3.3, and $pro(bc)$ = 1.45. Let α be 0.3, β be 0.3, and γ be 0.05. After comparing each value with the corresponding threshold, we have that $uo(b)$ is less than β. Thus, the itemset b is not a PHUOP.

Definition 6: Supposing that the items in the database are arranged in a certain order, such as alphabetical order or ascending order, then there is no harm in reordering the items in the database in support of ascending order and expressing this order with the symbol $\prec$.

Example 6: For example, in the above database, the support counts of each item can be easily obtained, and they are $SC(a)$: 5, $SC(b)$: 5, $SC(c)$: 8, $SC(d)$: 7, and $SC(e)$: 4. Since $SC(e) \leq SC(a) \leq SC(b) \leq SC(d) \leq SC(c)$ holds, the support-ascending order is $e \prec a \prec b \prec d \prec c$.

Problem Statement. The main goal of this paper is to first give an uncertain quantitative database and then discover interesting patterns that satisfy the conditions in which the support count is no less than the minimum support count threshold α, the utility occupancy value is no less than the minimum utility occupancy threshold β, and the probability

Authorized licensed use limited to: Rutgers University. Downloaded on May 20,2021 at 01:59:59 UTC from IEEE Xplore. Restrictions apply.

value that exists in the database is equal or greater than the minimum probability threshold γ.

III. Proposed Algorithm for Mining PHUOPs

A. Two data structures

Inspired by the idea of utility-list structure [18], this paper proposes two list-based structures to store the necessary information for mining potential high utility-occupancy patterns. The two list-based structures are described in detail below.

Definition 7 (PUO-list): The probability utility occupancy list (PUO-list) is a collection of tuples where an itemset X appears. It includes four elements (tid, pro, uo, ruo), which are an identifier of the transaction (tid), the probability value (pro), the utility occupancy value (uo), and the remaining utility occupancy value (ruo) of itemset (X). Among them, (pro) is the occurrence probability of itemset (X), (uo) is the proportion of utility in the transaction, and (ruo) in the given order database is the proportion of sum of the utilities of all the items after itemset (X) in this transaction. Note that the details of *ruo* concept can be referred to [12].

Example 7: For example, each item in every transaction is first reordered in support count ascending order. Then, a PUO-list using itemset (e) as an example can be considered, where (e) appears in transactions T_5, T_6, T_8, and T_{10}. The probability of (e) is 0.8, the utility occupancy is 0.1837, and the remaining utility occupancy is 0.8163, which appears in transaction T_5. Consequently, one tuple in the PUO-list of (e) can be written as $(T_5, 0.8, 0.1837, 0.8163)$. The other tuples of (e) are then calculated and listed in the same way. Finally, the PUO-lists of (e) are listed in the left of Fig. 1.

For easier calculation, the necessary information can be extracted from the designed PUO-list and put into the PFU-table, as defined below.

Definition 8 (PFU-table): The information in the probability-frequency-utility table (PFU-table) can be extracted from the PUO-list. Note that the probability of an itemset (X) is the sum of probabilities in each transaction that contains it, and the average utility occupancy of an itemset (X) is equal to the average of effective utility occupancy in the corresponding PUO-list. Similarly, the average remaining utility occupancy is equal to the average of all remaining utility occupancy of (X).

Example 8: Observing the PUO-list of itemset (e) in Fig. 1, it can be seen that (e) appears in four transactions, and thus its support count is 4 and the sum of the probability of (e) appearing in these four transactions is (0.8 + 0.8 + 0.5 + 0.7) = 2.8. The calculation process of its utility occupancy is (0.1837 + 0.6207 + 0.5902 + 0.2143)/4 = 0.4022, and its remaining utility occupancy is (0.8163 + 0.379 3 + 0.4098 + 0.7857)/4 = 0.5978. Finally, the construction process of (e) is shown in Fig. 1.

When the PUO-lists and PFU-tables of the all 1-itemsets have been constructed, there is no need to follow these processes to build them for k-itemsets $(k > 1)$ by rescanning the database. Algorithm 1 shows how to calculate k-itemsets

(*e*)			
tid	***pro***	***uo***	***ruo***
T_5	0.8	0.1837	0.8163
T_6	0.8	0.6207	0.3793
T_8	0.5	0.5902	0.4098
T_{10}	0.7	0.2143	0.7857



(*e*)			
sup	***pro***	***uo***	***ruo***
4	2.8	0.4022	0.5978

Fig. 1: The PUO-list and PFU-table of itemset (e)

Algorithm 1 Construction(X, X_a, X_b)

Input: X, an itemset with its corresponding *PUO-list* and *PFU-table*; X_a, the extension of X with an item a; X_b, the extension of X with an item b.
Output: X_{ab}.

```
1: initialize X_ab.PUO ← ∅, X_ab.PFU ← ∅;
2: set supUB = X_a.PFU.sup;
3: for each tuple E_a ∈ X_a.PUO do
4:     if ∃E_a ∈ X_b.PUO ∧ E_a.tid == E_b.tid then
5:         if X.PUO ≠ ∅ then
6:             search for E ∈ X.PUO, E.tid = E_a.tid;
7:             E_ab ← <E_a.tid, E_a.pro × E_b.pro/E.pro,
   E_a.uo + E_b.uo - E.uo, E_b.ruo>;
8:             X_ab.PFU.pro += E_a.pro × E_b.pro/E.pro;
9:             X_ab.PFU.uo += E_a.uo + E_b.uo - E.uo;
10:            X_ab.PFU.ruo += E_b.ruo;
11:        else
12:            E_ab ← <E_a.tid, E_a.pro × E_b.pro, E_a.uo +
   E_b.uo, E_b.ruo>;
13:            X_ab.PFU.pro += E_a.pro × E_b.pro;
14:            X_ab.PFU.uo += E_a.uo + E_b.uo;
15:            X_ab.PFU.ruo += E_b.ruo;
16:        X_ab.PUO ← X_ab.PUO ∪ E_ab;
17:        X_ab.PFU.sup ++;
18:    else
19:        supUB - -;
20:        if supUB < α × |D| then
21:            return null;
22: return X_ab
```

$(k \geq 2)$ based on 1-itemsets. At the beginning, an itemset X and two extensions of it, namely X_a and X_b, are given, and the order of a precedes b. A new itemset X_{ab} can be obtained by combining these two extensions. The proposed algorithm also involves a pruning strategy, which is explained in detail in the next subsection.

B. Upper bound on probability and utility occupancy

It is well-known that the downward closure property (aka Apriori property [1]) is not applicable for utility occupancy. In general, constructing the two structures of all the itemsets requires much memory and runtime. Consequently, it is necessary to find the upper bound on utility occupancy, which is denoted as $\hat{\phi}$.

Definition 9 (SC-tree): According to a previous study [19], a set-enumeration tree can be constructed and enumerated in a certain order. In the UHUOPM algorithm, the order

Authorized licensed use limited to: Rutgers University. Downloaded on May 20,2021 at 01:59:59 UTC from IEEE Xplore. Restrictions apply.

of the ascending support count is taken as the overall order of the set-enumeration tree, and the full name is Support-Count tree (SC-tree).

Lemma 1: For any node/itemset X in the designed SC-tree, the upper bound on the utility occupancy of its child node/itemset Y can be calculated as $\hat{\phi} = \frac{\sum_{top\alpha\times|D|,T_k\in\Gamma_X}\{uo(X,T_k)+ruo(X,T_k)\}^{\downarrow}}{|\alpha\times|D||}$ [12], where $|D|$ denotes the number of transactions in the database and Γ_X is the collection of transactions that contain itemset X. Besides, top and $\downarrow$ signify that the values of the utility occupancy are sorted in descending order, and the top $\alpha\times|D|$ values are utilized for further calculation, in which k is the length of X (a k-itemset). The detailed proof of this upper bound can be referred to the prior work [12].

Example 9: For example, consider the node (c) and its two subsets, (ca) and (cd), in the SC-tree. The utility occupancy of (c) can be calculated by Definition 3 and the value is 0.6468. Similarly, according to Lemma 1, the upper bound on the subset with a size of 2 with (c) as the root node can be calculated as 0.3081. This upper bound is greater than the minimum threshold of utility occupancy.

Lemma 2: Suppose there exists itemset X_k (containing k items) and X_{k-1} (containing k-1 items) in an uncertain database, and X_{k-1} is a subset of X_k. If X_k is a high probability itemset, then X_{k-1} should be a high probability itemset too. In other words, the high probability itemset has a downward closure property, such as $pro(X_k) < pro(X_{k-1})$ [16].

Example 10: For example, the probability of node (c) appearing in the running example is 5.4 while that of node (ca) is 2.13. The former should be greater than or equal to the latter.

C. Proposed algorithm and pruning strategy

This section describes the main procedures of the proposed UHUOPM algorithm and the effective pruning strategies in detail.

Strategy 1: When depth-first traversing the designed SC-tree as mentioned above, if the support count of a node X is less than the user-defined minimum support threshold α multiplied by the database size, then this node and its descendants can be directly pruned.

Proof: This strategy is based on the Apriori algorithm [1], and the property can be extracted as $SC(X_k) \leq SC(X_{k-1})$. There is no doubt that if $SC(X_{k-1}) < \alpha\times|D|$, then $SC(X_k) < \alpha\times|D|$ and X_k can be directly pruned. ■

Strategy 2: In an SC-tree constructed in $\prec$ order, if the upper bound on utility occupancy of their offspring is calculated based on a node X, which is less than the user-defined minimum threshold β, then all the nodes rooted at X as descendant nodes can be quickly pruned.

Proof: After building the corresponding list structures for a tree node X in SC-tree, the upper bound on utility occupancy of X can be quickly calculated using Lemma 1. Since this value is derived from the utility occupancy and the remaining utility occupancy, if the upper bound is less than the minimum threshold β, then there is no need to build the PUO-lists of any descendant nodes of X. ■

Algorithm 2 UHUOPM(D, *utable*, α, β, γ)

Input: an uncertain transaction database D, utility table $utable$, the minimum support threshold α, the minimum utility occupancy threshold β, and the minimum probability threshold γ.

Output: *PHUOPs*.

1: scan D to calculate the $SC(i)$ and $pro(i)$ of each item $i \in I$, and the tu value of each transaction;
2: find $I^* \leftarrow \{i \in I | SC(i) \geq \alpha\times|D| \wedge pro(i)) \geq \gamma\times|D|\}$;
3: sort I^* in the designed total order $\prec$, such as ascending support count;
4: using the total order $\prec$, scan D once to build the PUO-list and PFU-table for each 1-item $i \in I^*$;
5: **call *PHUOP-Search***$(\phi, I^*, \alpha, \beta, \gamma)$.
6: **return** *PHUOPs*

Strategy 3: In the designed SC-tree, if the overall probability of a node/pattern X is greater than or equal to the minimum probability threshold γ, then this pattern is a high probability pattern. On the contrary, if the value is less than the threshold, then node X and all nodes with it as the root will be pruned.

Proof: Like Strategy 1, based on Lemma 2, we can obtain $pro(X_k) < pro(X_{k-1})$. It is easy to acquire that $pro(X_k) < \gamma$ in the case of $pro(X_{k-1}) < \gamma$. ■

Strategy 4: One step closer to Strategy 1 in Algorithm 1, if the support count that itemset X_a holds is less than or equal to the minimum support threshold $\alpha\times|D|$, then it is not necessary to measure its extended itemset X_{ab}.

Proof: The function of Strategy 4 is the same as Strategy 1, except that Strategy 4 strengthens the judgment at the end of the proposed UHUOPM algorithm. ■

Feasible strategies for trimming the search space and reducing the runtime are proposed above. The core processes of the UHUOPM algorithm are explained according to these proposed strategies and shown below.

For the UHUOPM algorithm, the processed database with its utility-table and three parameters are needed in advance. They are the uncertain quantitative database D, the unit utility corresponding to each item w.r.t. $utable$, the minimum support threshold α, the minimum utility occupancy threshold β, and the minimum probability threshold γ. At the beginning of Algorithm 2, during the first traversal of the database, the support count and corresponding probability for each item are calculated. At the same time, the transaction utility (*tu*) of the transactions are calculated according to Definition 2, which will be used in the subsequent processes. Then, the 1-itemsets I^* whose support count and probability meet the conditions are filtered out, and these itemsets in every processed transaction in the ascending order of their support counts are reordered. After that, the database is traversed again to construct the corresponding PUO-lists and PFU-table for each 1-itemset in I^*. After the initial processes, the next step is to filter out PHUOPs based on

Authorized licensed use limited to: Rutgers University. Downloaded on May 20,2021 at 01:59:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 PHUOP-Search(X, *extenOfX*, α, β, γ)

Input: an uncertain transaction database D, an itemset X and its extended itemsets *extenOfX*, the minimum support threshold α, the minimum utility occupancy threshold β, and the minimum probability threshold γ.
Output: *PHUOPs*.

1: **for** each itemset $X_a \in$ *extenOfX* **do**
2: obtain $SC(X_a)$ and $uo(X_a)$ from the built PFU-table w.r.t. $X_a.PFU$;
3: **if** $SC(X_a) \geq \alpha \times |D| \wedge pro(X_a) \geq \gamma \times |D|$ **then**
4: **if** $uo(X_a) \geq \beta$ **then**
5: *PHUOPs* $\leftarrow$ *PHUOPs* $\cup X_a$;
6: $\hat{\phi}(X_a) \leftarrow$ ***UpperBound***$(X_a.PUO, \alpha)$;
7: **if** $\hat{\phi}(X_a) \geq \beta$ **then**
8: *extenOfX*$_a \leftarrow \emptyset$;
9: **for** each $X_b \in$ *extenOfX* that $X_a \prec X_b$ **do**
10: $X_{ab} \leftarrow X_a \cup X_b$;
11: call ***Construct***(X, X_a, X_b);
12: **if** $X_{ab}.PUO \neq \emptyset \wedge SC(X_{ab}) \geq \alpha \times |D|$ **then**
13: *extenOfX*$_a \leftarrow$ *extenOfX*$_a \cup X_{ab}.PUO$;
14: **call *PHUOP-Search***(X_a,*extenOfX*$_a$, α, β,γ);
15: **return** *PHUOPs*

Algorithm 4 UpperBound($X_a.PUO$, α)

Input: an uncertain transaction database D, itemset X_a and its corresponding PUO-list, the minimum support threshold α.
Output: the upper bound on X_a, $\hat{\phi}(X_a)$.

1: *sumTopK* $\leftarrow 0, \hat{\phi}(X_a) \leftarrow 0, V_{occu} \leftarrow \emptyset$;
2: calculate $(uo(X_a, T_q) + ruo(X_a, T_q))$ of each tuple from the built $X_a.PUO$ and put them into the set of V_{occu};
3: sort V_{occu} by descending order as $V^{\downarrow}_{occu}$;
4: **for** $k \leftarrow 1$ to $\alpha \times |D|$ in $V^{\downarrow}_{occu}$ **do**
5: *sumTopK* $\leftarrow$ *sumTopK* + $V^{\downarrow}_{occu}[k]$;
6: $\hat{\phi}(X_a) = \dfrac{sumTopK}{\alpha \times |D|}$.
7: **return** $\hat{\phi}(X_a)$

the given conditions. More details are given in Algorithm 3. Strategy 4 is applied in lines 19 to 21 in Algorithm 1. The support count of the extension of X_a should be equal to or less than that of X_a. Using this condition, whether the extension of X_a can be directly trimmed is determined without calculating other conditions. Algorithm 4 develops the designed upper bound calculation formula obtained by Lemma 1.

IV. Experiments

This section describes the experiments that were conducted. To evaluate whether the proposed algorithm is acceptable, the proposed UHUOPM algorithm was compared with the HUOPM algorithm [12] in terms of runtime, which is a significant indicator to evaluate the pros and cons of an algorithm. We also included two variants in the comparison to further evaluate the performance of the proposed pruning strategies. We call these two algorithms UHUOPM$_1$ (using the pruning strategies 1 and 2) and UHUOPM$_2$ (using the pruning strategies 1 and 3).

All the experimental procedures were written in Java and the program was ran on a desktop computer. The computer's basic configuration included 4GB of memory and 64-bit Windows 7 operating system. To better evaluate the performance of the compared algorithms, this experiment involved not only realistic datasets but also synthesized datasets. Both real-life datasets (consisting of mushroom, retail, and kosarak) and the synthesized dataset (T40I10D100K) were selected for the experiments.

The runtime of the four algorithms is evaluated below. Since the algorithm involves in three parameters, experiments on the three ingredients needed to be performed separately. Here, we only give the analysis of the case where the support and probability vary respectively. The trends of the runtime under the conditions that the support, utility occupancy, and probability separately changed while the other two parameters were fixed are respectively shown in Fig. 2 and Fig. 3. Since the HUOPM algorithm is based on an precise database, it does not contain probability values.

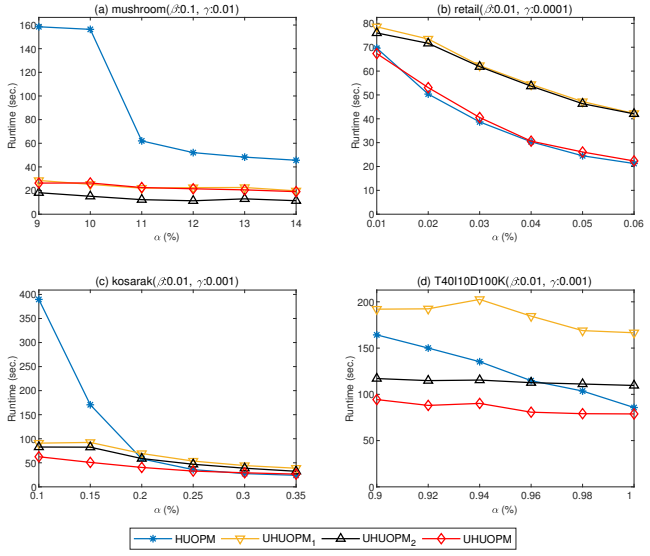


Fig. 2: Runtime under a changed α with a fixed β and γ

The UHUOPM algorithm was generally superior to the other three algorithms in runtime except for on the mushroom dataset. This implies that several of the strategies proposed in this paper worked well. For example, as shown in Fig. 2 (d), β was set to 0.01, γ was 0.001, and α changed from 0.9% to 1% in increments of 0.02%. The runtime of the other two versions of the proposed algorithm were between HUOPM and UHUOPM. This is due to the lack of probability constraints in HUOPM. The number of traversed nodes was much more than the other algorithms, and thus the time consumption was significantly large. Compared to the UHUOPM algorithm, UHUOPM$_1$ and UHUOPM$_2$ were not good enough because

Authorized licensed use limited to: Rutgers University. Downloaded on May 20,2021 at 01:59:59 UTC from IEEE Xplore. Restrictions apply.

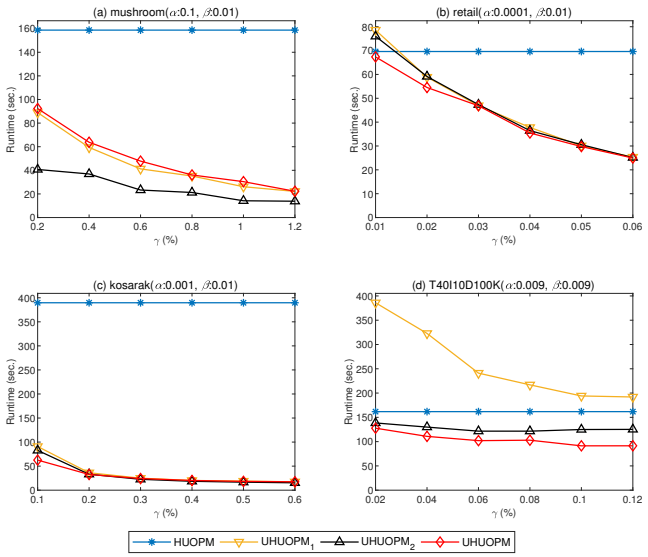


Fig. 3: Runtime under a changed γ with a fixed α and β

they used part of the proposed strategies. They traversed more nodes, and thus their runtimes were slightly more than the UHUOPM algorithm. As shown in Fig. 2(a), β was set to 0.1, γ was 0.01, and α changed from 9% to 14% in increments of 1%. This figure shows that the runtime of the UHUOPM$_2$ algorithm was the shortest. This is because the mushroom dataset is very dense, and Strategy 1 and Strategy 3 play an obvious role while Strategy 2 and Strategy 4 had little effect. In this case, probability constraint plays an important role in the mining process, which speeds up the pruning ratio. A similar situation occurred when the utility occupancy was fixed or the probability was fixed, as shown in the other datasets. Besides, Fig. 3 depicts the condition in the runtime when the other parameters were fixed and the probability varied. No matter how the minimum probability threshold varied, the runtime of the HUOPM algorithm was always stable. This is because the test datasets involved in the HUOPM algorithm were precise instead of uncertain. In other words, all the occurrence probabilities of the itemsets processed by HUOPM were 1.0; thus, its image curve was reasonably a straight line.

V. Conclusion and Future Work

In this paper, a novel algorithm namely UHUOPM was proposed for mining high utility-occupancy pattern in uncertain databases. The proposed algorithm adopts a novel mining framework that uses two list-based structures to reduce database traversal. Several effective pruning strategies were utilized to improve the efficiency of searching and reduce the running time. Follow-up experiments were conducted to analyze the performance of the compared algorithms in terms of runtime. Since this is the first work to find PHUOPs in an uncertain database, there is still much room for future research in terms of different constraint-based patterns or other types of processed data (e.g., sequence [20], complex event [21], graph, etc).

References

[1] R. Agrawal, R. Srikant *et al.*, "Fast algorithms for mining association rules," in *Proceedings of the 20th International Conference on Very Large Data Bases*, 1994, pp. 487–499.

[2] W. Gan, J. C. W. Lin, H. C. Chao, and J. Zhan, "Data mining in distributed environment: a survey," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 6, 2017.

[3] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, and P. S. Yu, "A survey of parallel sequential pattern mining," *ACM Transactions on Knowledge Discovery from Data*, vol. 13, no. 3, pp. 1–34, 2019.

[4] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, V. S. Tseng, and P. S. Yu, "A survey of utility-oriented pattern mining," *IEEE Transactions on Knowledge and Data Engineering. DOI: 10.1109/TKDE.2019.2942594*, 2019.

[5] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Weighted frequent itemset mining over uncertain databases," *Applied Intelligence*, vol. 44, no. 1, pp. 232–250, 2016.

[6] R. Chan, Q. Yang, and Y. D. Shen, "Mining high utility itemsets," in *Proceedings of the 3rd IEEE International Conference On Data Mining*. IEEE, 2003, pp. 19–26.

[7] H. Yao and H. J. Hamilton, "Mining itemset utilities from transaction databases," *Data and Knowledge Engineering*, vol. 59, no. 3, pp. 603–626, 2006.

[8] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, T. P. Hong, and H. Fujita, "A survey of incremental high-utility itemset mining," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 8, no. 2, p. e1242, 2018.

[9] Y. Liu, W. K. Liao, and A. Choudhary, "A two-phase algorithm for fast discovery of high utility itemsets," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2005, pp. 689–695.

[10] L. Tang, L. Zhang, P. Luo, and M. Wang, "Incorporating occupancy into frequent pattern mining for high quality pattern recommendation," in *Proceedings of the 21st ACM International Conference on Information and Knowledge Management*. ACM, 2012, pp. 75–84.

[11] B. Shen, Z. Wen, Y. Zhao, D. Zhou, and W. Zheng, "Ocean: fast discovery of high utility occupancy itemsets," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2016, pp. 354–365.

[12] W. Gan, J. C. W. Lin, P. Fournier-Viger, H. C. Chao, and P. S. Yu, "HUOPM: High utility occupancy pattern mining," *IEEE Transactions on Cybernetics*, vol. 50, no. 3, pp. 1195–1208, 2020.

[13] C. C. Aggarwal, Y. Li, J. Wang, and J. Wang, "Frequent pattern mining with uncertain data," in *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2009, pp. 29–38.

[14] C. K. Chui, B. Kao, and E. Hung, "Mining frequent itemsets from uncertain data," in *Pacific-Asia Conference on knowledge discovery and data mining*. Springer, 2007, pp. 47–58.

[15] C. K. S. Leung, M. A. F. Mateo, and D. A. Brajczuk, "A tree-based approach for frequent pattern mining from uncertain data," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2008, pp. 653–661.

[16] J. C. W. Lin, W. Gan, P. Fournier-Viger, T. P. Hong, and V. S. Tseng, "Efficient algorithms for mining high-utility itemsets in uncertain databases," *Knowledge-Based Systems*, vol. 96, pp. 171–187, 2016.

[17] J. Han, J. Pei, Y. Yin, and R. Mao, "Mining frequent patterns without candidate generation: A frequent-pattern tree approach," *Data Mining And Knowledge Discovery*, vol. 8, no. 1, pp. 53–87, 2004.

[18] M. Liu and J. Qu, "Mining high utility itemsets without candidate generation," in *Proceedings of the 21st ACM International Conference on Information and Knowledge Management*. ACM, 2012, pp. 55–64.

[19] R. Rymon, "Search through systematic set enumeration," *Proceeding of the 3rd International Conference on Principles of Knowledge Representation and Reasoning*, pp. 539–550, 1992.

[20] W. Gan, J. C. W. Lin, J. Zhang, H. C. Chao, H. Fujita, and P. S. Yu, "ProUM: High utility sequential pattern mining," in *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics*. IEEE, 2019, pp. 767–773.

[21] W. Gan, J. C. W. Lin, H. C. Chao, and P. S. Yu, "Utility-driven mining of high utility episodes," in *Proceedings of the IEEE International Conference on Big Data*. IEEE, 2019, pp. 2644–2653.

Authorized licensed use limited to: Rutgers University. Downloaded on May 20,2021 at 01:59:59 UTC from IEEE Xplore. Restrictions apply.